

Time-series forecasting and sequence models

Look at the series, beat the simple baselines, then model the lags: autoregression and recurrent networks.

THE QUESTION

Given the past values of a series, how do we forecast the next ones, and how do we know the forecast is any good?

THE INTUITION

The past carries the signal: a trend, a season and the pull of recent values. Start from simple rules such as "same as last month", split in time, and score every model against them. Autoregression regresses a value on its own lags; a recurrent network learns its own summary of the past.

KEY TAKEAWAY

Split in time, never at random, and always report a naive or seasonal naive baseline. A model that cannot beat it has learnt nothing useful.

plot the series → trend, season, noise → split in time → baselines → MAE, RMSE, MAPE → difference → AR(p) or RNN

1 Some series are easier to forecast

	electricity demand	stock price
we understand the drivers	yes: temperature	no
plenty of past data	yes	yes
forecast leaves the outcome alone	yes	no: the market reacts
verdict	forecastable	hard

Predictability depends on how well we understand the drivers, how much data there is, and whether the forecast changes the outcome. Models for demand E_T : **explanatory** $f(\text{temp}_T, \text{hour}_T, \dots)$, **time series** $f(E_{T-1}, E_{T-2}, \dots)$, or **mixed** (both).

2 Trend, season and what is left



Trend: a long-term rise or fall. **Season:** a pattern with a fixed period, such as the month or the weekday. **Cycle:** rises and falls with no fixed period. Our 120 months (illustrative) have slope 0.5, a yearly season and noise sd 2.4; a centred 2×12 moving average recovers the slope. With none of these a series is close to noise and hard to forecast.

3 Four baselines need no training



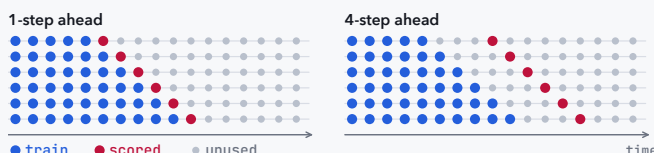
Forecast $\hat{y}_{T+h|T}$ from $y_1, \dots, y_T$: **mean** $\frac{1}{T} \sum_t y_t$; **naive** y_T ; **seasonal naive** $y_{T-h-m(k+1)}$ with period m and $k = \lfloor (h-1)/m \rfloor$; **drift** $y_T + h(y_T - y_1)/(T-1)$, the textbook's fourth. Monthly data, now January, forecast March: $h = 2$, $m = 12$, $k = 0$, so $\hat{y} = y_{T-10}$, last March.

4 Score forecasts on the future you held back

on the 24 test months	MAE	RMSE	MAPE %	MAE
drift	6.55	7.46	5.86	
seasonal naive	8.21	9.43	7.18	
naive	8.50	9.83	7.33	
mean	29.55	30.45	25.68	

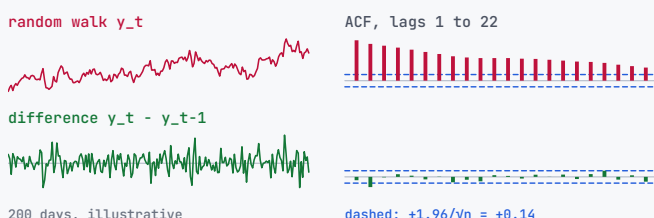
Train on months 1 to 96, test on 97 to 120. With $e_t = y_t - \hat{y}_t$: $\text{MAE} = \text{mean}|e_t|$, $\text{RMSE} = \sqrt{\text{mean } e_t^2}$, $\text{MAPE} = 100 \cdot \text{mean}|e_t/y_t|$, which breaks near $y = 0$. Seasonal naive misses a year of trend ($12 \times 0.5 = 6$), drift misses the season, the mean misses both. Simple methods often work well, more so when you know the domain.

5 Time-series CV moves the origin forward



Each row trains on everything up to an origin and scores the forecast h steps later; average over origins. Random folds would train on the future (see the cross-validation sheet); to tune, split each training block again in time. Over 48 origins the 1-step MAE is 4.99 for naive, 6.06 for seasonal naive; at 4 steps it is 11.15 against 5.95.

6 Stationary series look the same at any time



White noise is stationary; a trend or a season is not; cycles with no fixed period can be. With $r_k = \sum_t (y_t - \bar{y})(y_{t+k} - \bar{y}) / \sum_t (y_t - \bar{y})^2$, our random walk (like the slides' stock series) has $r_1 = 0.91$ and 0.30 at lag 22. Its first difference is close to white noise: 2 of 22 lags cross the band.

From lags to hidden states

Difference the series, regress on its lags, then let a recurrent network learn its own summary of the past.

7 Differencing removes trend and season

y_t : trend and season

$y_t - y_{t-1}$: trend gone, season stays

$y_t - y_{t-12}$: season gone, mean 6 = 12×0.5

The first difference $y_t - y_{t-1}$ turns a linear trend into a constant (mean 0.46) but keeps the season: its lag-12 autocorrelation is 0.64. The seasonal difference $y_t - y_{t-m}$ removes the season and leaves 12×0.5 (mean 6.04). The slides show the same with a stock series.

8 Autoregression is linear regression on lags

6.49	y_1	6.93	y_2	7.51	y_3	8.49	y_4	9.00	y_5	11.40	y_6
1	y_{t-1}	y_{t-2}				y_t					
1	6.93	6.49				7.51					
1	7.51	6.93				8.49					
1	8.49	7.51				9.00					

X: 498 rows $\times$ 3

least squares

	c	φ_1	φ_2
true	1.00	0.60	0.30
fit	1.34	0.57	0.28

target

$$y_t = c + \varphi_1 y_{t-1} + \dots + \varphi_p y_{t-p} + \varepsilon_t, \quad \varepsilon_t \text{ white noise}$$

Each row of X is $[1, y_{t-1}, \dots, y_{t-p}]$ and the target is y_t : solve least squares, as in the linear regression sheet. Our AR(2) with $c = 1, \varphi = (0.6, 0.3)$, 500 values, fits (1.34, 0.57, 0.28). The notebook (quoted): 10 lags on monthly CO₂, test MSE 0.210; 20 lags on airline passengers, RMSE 18.8.

9 Recursive forecasts feed on themselves



One-step forecasts use the true lags. Going h steps ahead feeds each forecast back as a lag, so errors compound and the forecast decays to the mean $c/(1 - \varphi_1 - \varphi_2)$. By $h = 4$ its MAE (1.17) nearly equals the mean's (1.20). A regular pattern holds up: on the monthly series AR(13) scores test MAE 2.45 one-step, 1.94 recursive (drift 6.55). The notebook scores one-step only.

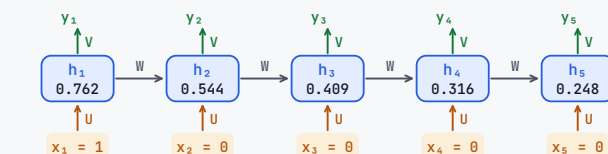
13 Pause and think

In January, seasonal naive for March? Last March.
Why is a stock price hard?
Forecasts move the market.

Shuffle a series into 5 folds? No: it would train on the future.
AR(2), 24 steps ahead? About the fitted mean.

Is a random walk stationary? No; its first difference is.
Why do LSTMs remember longer?
The cell state is added to.

10 An RNN carries a hidden state

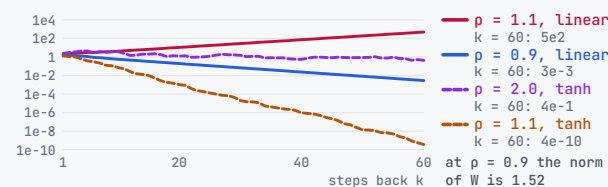


illustrative: $W = 0.8, U = 1.0, b = 0$; one input pulse, then zeros

$$h_t = \tanh(W h_{t-1} + U x_t + b), \quad y_t = V h_t + c$$

The same weights act at every step, so any length works; h_t is the memory (W_{hh}, W_{xh}, W_{hy} in the slides). **Many-to-one**: a review in, sentiment out. **Many-to-many**: an encoder reads French, a decoder writes English. Training: backprop through time.

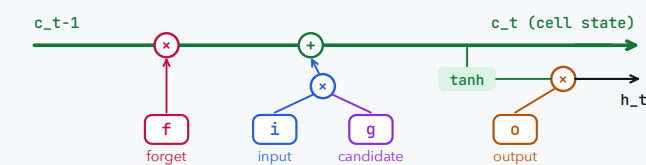
11 Gradients through time multiply Jacobians



$$\frac{\partial h_T}{\partial h_{T-k}} = \prod_{j=T-k+1}^T \text{diag}(1 - h_j^2) W$$

32 units, W scaled to spectral radius ρ . Without $\tanh'$, at $k = 60$ the norm is 0.003 for $\rho = 0.9$ and 487 for $\rho = 1.1$; $\tanh' \leq 1$ shrinks it further. The rate is set by ρ , not $\|W\|$: $\|W\|_2 = 1.52$ at $\rho = 0.9$, yet it vanishes.

12 LSTM and GRU gate the memory



each gate reads $[h_{t-1}, x_t]$; f, i, o use σ (0 to 1), g uses $\tanh$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \quad h_t = o_t \odot \tanh(c_t)$$

Along c the gradient is scaled by f_t , not by W and $\tanh'$, so with f_t near 1 it survives. A **GRU** has 2 gates (update, reset) and often does as well. Transformers took over language; RNNs still suit streaming data (constant memory per step).

split in time · beat the naive baselines · difference to stationarity · AR = least squares on lags · gate the memory

Before: the naive Bayes and Bayesian networks sheet. This is the last sheet of the series; back to the start: the conventions and metrics sheet. Source: lecture slides "Forecasting" (after the textbook "Forecasting: Principles and Practice") and "Recurrent neural networks", and the autoregressive model notebook (nipunbatra.github.io/ml-teaching); CO₂ and airline numbers are its saved outputs, the rest is seeded NumPy. Corrected: the seasonal naive example ($k = 0$, so March 2019); the beer forecast's band is a prediction interval; $\|W_{hh}\| > 1$ alone does not make gradients explode; gates ease vanishing gradients but do not remove them.